

Modelagem Geométrica: Representações Celulares, Ray Tracing Volumétrico e Dinâmica de Partículas em Tempo Real

Motor de Cena 2D com Estruturas de Aceleração Espacial

Richard Elias Soares Viana
Herivelton Siqueira
IMPA Tech — Graduação em Matemática Aplicada

Maio de 2026

Sumário

1	Introdução	4
1.1	Organização do Relatório	4
2	Funções Implícitas e o Problema de Inclusão	4
2.1	Definição Formal	4
2.2	Exemplos Concretos	5
2.3	Mapeamento de Coordenadas	5
3	Discretização Celular e Classificação	6
3.1	Motivação para a Discretização	6
3.2	Grade Uniforme	6
3.2.1	Classificação de Células	7
3.3	Classificação Exata para Círculos	7
4	Quadtree Adaptativa	8
4.1	Motivação para a Adaptatividade	8
4.2	Definição Recursiva	8
4.3	Algoritmo de Construção	9
4.3.1	Complexidade	9
4.4	Poda (Pruning)	10
5	Ray Tracing Volumétrico	10
5.1	Modelo de Iluminação e Motivação	10
5.2	Equação do Raio	11
5.3	Interseção Raio–AABB: O Método das Lâminas	11
5.4	Travessia Hierárquica da Quadtree	12
5.5	Raymarching na Fronteira	12
5.6	Deteção de Imersão e Iluminação Invertida	13
5.7	Modelo de Decaimento de Luz	13
5.8	Visualização do Modo Adaptativo	14

6	Física de Partículas e Colisões Implícitas	14
6.1	Modelo de Movimento	14
6.2	Colisão com Formas Implícitas: Reflexão Pelo Gradiente	14
6.3	Sombras Dinâmicas por Shadow Ray	15
7	Análise de Complexidade e Performance	16
7.1	Custo do Ray Tracing por Frame	16
7.2	Vantagem da Poda	16
8	Integração do Sistema: o Loop Principal	16
8.1	Arquitetura de Módulos	16
8.2	Sequência de Operações por Frame	16
8.3	Evento <code>rebuild()</code>	17
9	Funções Customizadas: da String ao Código	17
10	Conclusão	18
A	Pseudocódigo Completo do Loop Principal	18
B	Tabela de Parâmetros do Sistema	19

Resumo

Este relatório detalha a concepção, a fundamentação matemática e a implementação de um sistema interativo de modelagem geométrica bidimensional. O sistema integra três pilares computacionais: *(i)* representações celulares de formas implícitas via grades uniformes e Quadrees adaptativas; *(ii)* um motor de iluminação volumétrica baseado em *Ray Tracing* acelerado por estrutura espacial hierárquica; e *(iii)* um sistema de partículas físicas com colisões implícitas e cálculo de sombras em tempo real. Cada componente é apresentado partindo da motivação matemática, passando pela dedução rigorosa dos algoritmos e chegando até os detalhes concretos de implementação em JavaScript. O objetivo central é demonstrar como conceitos de geometria diferencial, álgebra linear e análise numérica se traduzem diretamente em código eficiente e visualmente rico. Todo o conteúdo produzido nesse projeto se encontra no github: https://github.com/Heri-impatech/projeto-cg-impatech_oficial.

1 Introdução

A representação de formas geométricas no computador é um problema que admite abordagens radicalmente diferentes dependendo das operações que se deseja realizar. Modelos paramétricos — como splines de Bézier ou NURBS — descrevem a *fronteira* de uma forma de maneira explícita e contínua, sendo naturais para renderização e animação. No entanto, quando a pergunta central é “*um ponto p está dentro ou fora desta forma?*”, os modelos paramétricos exigem testes geométricos custosos.

As **superfícies implícitas** invertem essa lógica: uma forma é definida pelo conjunto de zeros de uma função $f : \mathbb{R}^2 \rightarrow \mathbb{R}$,

$$\mathcal{S} = \{(x, y) \in \mathbb{R}^2 \mid f(x, y) = 0\},$$

e o interior da forma é o conjunto $\{f(x, y) \leq 0\}$. Com isso, o teste de inclusão de um ponto se reduz a avaliar f naquele ponto — uma operação de tempo constante $O(1)$.

Este projeto explora como construir, sobre esse alicerce matemático, um sistema computacional completo: discretizamos o espaço em células, construímos uma hierarquia adaptativa (Quadtree), e usamos essa hierarquia para acelerar dois sistemas físicos acoplados — iluminação por *Ray Tracing* e dinâmica de partículas com colisões. O resultado é um ambiente interativo que roda em tempo real no navegador, onde o usuário pode manipular as formas implícitas e observar instantaneamente os efeitos sobre a luz e as partículas.

1.1 Organização do Relatório

O documento segue a mesma progressão lógica do código:

- §1. **Funções Implícitas e o Problema de Inclusão** — fundamentos matemáticos das formas utilizadas.
- §2. **Discretização Celular e Classificação** — como dividir o espaço em células e classificá-las em relação a uma forma.
- §3. **Quadtrees Adaptativas** — construção, subdivisão e poda da estrutura de aceleração espacial.
- §4. **Ray Tracing Volumétrico** — modelo de iluminação, travessia hierárquica e raymarching na fronteira.
- §5. **Física de Partículas** — integração de movimento, colisão implícita e sombras dinâmicas.
- §6. **Integração do Sistema** — como os módulos interagem no loop principal de renderização.

2 Funções Implícitas e o Problema de Inclusão

2.1 Definição Formal

Definição 2.1 (Superfície Implícita 2D). Seja $f : \Omega \subset \mathbb{R}^2 \rightarrow \mathbb{R}$ uma função contínua definida em um domínio aberto Ω . A *superfície implícita* (ou curva implícita, em duas dimensões) associada a f é o conjunto de nível

$$\mathcal{S}_f = \{(x, y) \in \Omega \mid f(x, y) = 0\}.$$

Além disso, definimos:

$$\begin{aligned} \mathcal{I}_f &= \{(x, y) \in \Omega \mid f(x, y) < 0\} && \text{(interior)} \\ \mathcal{E}_f &= \{(x, y) \in \Omega \mid f(x, y) > 0\} && \text{(exterior)} \end{aligned}$$

A convenção de que o *interior* corresponde a $f < 0$ é padrão em geometria implícita e se alinha com a noção de que f mede uma espécie de “distância com sinal” à superfície (embora não seja, em geral, a distância euclidiana).

Motivação

Por que usar funções implícitas em vez de polígonos? Para um polígono com n vértices, o teste clássico de ponto-interior (*ray casting* ou *winding number*) tem complexidade $O(n)$. Para uma função implícita, o teste é sempre $O(1)$: basta avaliar $f(p)$ e verificar o sinal. Isso é fundamental para sistemas que precisam testar *milhares* de pontos por frame, como o nosso motor de física e iluminação.

2.2 Exemplos Concretos

Círculo de centro (c_x, c_y) e raio r :

$$f(x, y) = (x - c_x)^2 + (y - c_y)^2 - r^2.$$

O interior $f < 0$ é exatamente o disco aberto de raio r . Note que $\nabla f = 2(x - c_x, y - c_y)$, um vetor que aponta radialmente para fora do centro — o que será crucial para o cálculo de normais na física de partículas.

Parábola $y = x^2$ como fronteira:

$$f(x, y) = y - x^2.$$

O interior $f < 0$ é a região *abaixo* da parábola, $\{y < x^2\}$.

Combinação de objetos (União Implícita): Dado um conjunto de formas $f_1, f_2, \dots, f_k$, a função

$$f_{\text{union}}(x, y) = \min_i f_i(x, y)$$

representa a **união** de todas as formas. Um ponto está dentro da união se e somente se está dentro de *ao menos uma* das formas, o que corresponde exatamente a $\min_i f_i(p) \leq 0$. Esta é a operação implementada na classe **Scene** do projeto:

```
1 class Scene {
2     evaluate(px, py) {
3         let minVal = Infinity;
4         for (let obj of this.objects)
5             minVal = Math.min(minVal, obj.evaluate(px, py));
6         return minVal;
7     }
8 }
```

2.3 Mapeamento de Coordenadas

O canvas do navegador usa um sistema de coordenadas em *pixels*, com a origem no canto superior esquerdo e o eixo y apontando *para baixo*. As funções matemáticas clássicas, por outro lado, são naturalmente definidas em coordenadas cartesianas com y apontando para cima. O sistema realiza um mapeamento afim bijetivo entre esses dois espaços.

Seja um canvas de largura W e altura H pixels. O domínio matemático escolhido é $[-2, 2] \times [-2, 2]$. O mapeamento de pixel (p_x, p_y) para coordenada matemática (x, y) é:

$$x = -2 + p_x \cdot \frac{4}{W} \quad (1)$$

$$y = -2 + (H - p_y) \cdot \frac{4}{H} \quad (2)$$

O termo $(H - p_y)$ na equação (2) é responsável por inverter o eixo y , convertendo a convenção “ y cresce para baixo” do canvas para a convenção matemática “ y cresce para cima”. A implementação direta aparece em `geometry.js`:

```

1 class CustomImplicit {
2   constructor(canvasW, canvasH, exprStr) {
3     this.scaleX = 4.0 / canvasW; // 4 = largura do dominio [-2,2]
4     this.scaleY = 4.0 / canvasH;
5     this.func = new Function('x', 'y', 'return ${exprStr};');
6   }
7   pxToDomainX(px) { return -2.0 + px * this.scaleX; }
8   pxToDomainY(py) { return -2.0 + (this.canvasH - py) * this.scaleY; }
9   evaluate(px, py) {
10     return this.func(this.pxToDomainX(px), this.pxToDomainY(py));
11   }
12 }

```

Atenção

O círculo interativo (adicionado com clique do mouse) usa uma escala diferente: seu domínio é $[0, 1] \times [0, 1]$ em vez de $[-2, 2] \times [-2, 2]$, pois é criado diretamente em coordenadas normalizadas de canvas. O raio padrão $r = 0.08$ corresponde a 8% da largura do canvas.

3 Discretização Celular e Classificação

3.1 Motivação para a Discretização

Motivação

Funções implícitas são objetos contínuos e infinitamente detalhados. Para *renderizá-las* e *consultar* se um raio as atravessa, precisamos de uma representação finita e discreta. A ideia de discretizar o espaço em células é análoga à integração numérica: substituímos um objeto contínuo por uma aproximação em malha, com resolução controlável. A chave é que, mesmo que uma célula não seja pequena o suficiente para capturar a fronteira exatamente, ela pode ser *classificada* de forma conservadora — o que é suficiente para a aceleração espacial.

3.2 Grade Uniforme

A abordagem mais simples consiste em dividir o domínio $[0, W] \times [0, H]$ em uma grade regular de $n \times n$ células, cada uma com dimensões $\Delta x = W/n$ e $\Delta y = H/n$. A célula na posição (i, j) ocupa a região

$$C_{ij} = [i\Delta x, (i+1)\Delta x] \times [j\Delta y, (j+1)\Delta y].$$

3.2.1 Classificação de Células

O **status** de uma célula descreve sua relação com a forma implícita:

Status	Nome	Cor	Significado
0	Exterior	Branco	$f > 0$ em toda a célula
1	Interior	Azul	$f \leq 0$ em toda a célula
2	Fronteira	Vermelho	f muda de sinal na célula

A determinação rigorosa do status exigiria verificar f em *todos* os pontos da célula — um problema inviável. Na prática, usamos uma **heurística de amostragem** nos 4 cantos e no centro da célula:

Definição 3.1 (Classificação por Amostragem). Seja $P = \{(x_{\min}, y_{\min}), (x_{\max}, y_{\min}), (x_{\max}, y_{\max}), (x_{\min}, y_{\max})\}$ o conjunto dos 4 cantos e do centro de uma célula. Definimos:

$$k = \#\{p \in P \mid f(p) \leq 0\}.$$

Então:

$$\text{status}(C) = \begin{cases} 1 & \text{se } k = 5 \text{ (todos dentro)} \\ 0 & \text{se } k = 0 \text{ (todos fora)} \\ 2 & \text{caso contrário (fronteira estimada)} \end{cases}$$

Observação 3.2. Esta heurística pode errar em casos extremos: uma forma muito pequena que toque o interior de uma célula sem passar por nenhum dos 5 pontos amostrados seria classificada como Status 0 (exterior), quando na verdade deveria ser Status 2. Na prática, para formas suaves e resoluções razoáveis de grade, isso não é um problema. Para círculos, existe uma classificação *exata* baseada na distância do centro ao retângulo mais próximo, que é implementada na classe **Circle** (ver Seção 3.3).

3.3 Classificação Exata para Círculos

Para o círculo de centro (c_x, c_y) e raio r , é possível determinar o status de uma célula de forma *exata* sem recorrer a amostragem. A ideia é calcular a distância mínima do centro do círculo ao retângulo (célula).

Proposição 3.3 (Classificação Exata do Círculo). Seja C uma célula com limites $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$. O ponto do retângulo C mais próximo do centro (c_x, c_y) é

$$q = (\text{clamp}(c_x, x_{\min}, x_{\max}), \text{clamp}(c_y, y_{\min}, y_{\max})),$$

onde $\text{clamp}(v, a, b) = \max(a, \min(b, v))$.

Então:

1. Se $\|q - (c_x, c_y)\|^2 > r^2$: a célula é **exterior** (Status 0). O círculo não alcança a célula.
2. Se todos os 4 cantos satisfazem $(p_x - c_x)^2 + (p_y - c_y)^2 < r^2$: a célula é **interior** (Status 1).
3. Caso contrário: a célula é **fronteira** (Status 2).

Demonstração. (1) Se o ponto mais próximo do retângulo ao centro c está fora do círculo, então todo o retângulo está fora, pois todos os demais pontos do retângulo estão ainda mais distantes de c .

(2) Se todos os cantos estão dentro do círculo e o raio do círculo é maior que a distância de c ao retângulo, então, como os cantos são os extremos do retângulo convexo, todos os pontos intermediários também estão dentro.

(3) É um caso misto: o círculo intercepta, mas não contém completamente o retângulo. $\square$

A implementação em JavaScript:

```
1 classifyCell(xMinPx, xMaxPx, yMinPx, yMaxPx) {
2     // Converte para coordenadas de dominio (normalizado 0..1)
3     const xMin = this.pxToDomainX(xMinPx), xMax = this.pxToDomainX(xMaxPx);
4     const yMin = this.pxToDomainY(yMinPx), yMax = this.pxToDomainY(yMaxPx);
5
6     // Passo 1: ponto mais proximo do centro no retangulo
7     const closestX = clamp(this.cx, xMin, xMax);
8     const closestY = clamp(this.cy, yMin, yMax);
9     const dx = closestX - this.cx, dy = closestY - this.cy;
10
11     // Se o ponto mais proximo esta fora do circulo => Status 0
12     if (dx*dx + dy*dy > this.rSq) return 0;
13
14     // Verifica se TODOS os cantos estao dentro
15     const corners = [[xMin,yMin],[xMax,yMin],[xMax,yMax],[xMin,yMax]];
16     for (let c of corners)
17         if ((c[0]-this.cx)**2 + (c[1]-this.cy)**2 >= this.rSq) return 2;
18
19     return 1; // Todos os cantos dentro => Status 1
20 }
```

4 Quadtree Adaptativa

4.1 Motivação para a Adaptatividade

Motivação

Uma grade uniforme de resolução n possui n^2 células. Para $n = 512$ (o que daria fronteiras com precisão de 1 pixel em um canvas de 512px), teríamos 262.144 células. A maioria estaria classificada como Status 0 (exterior) ou Status 1 (interior) — informação quase inútil para o Ray Tracer, que só precisa saber *onde estão as fronteiras*. A Quadtree resolve isso: ela refina *somente* as células de fronteira (Status 2), concentrando o poder computacional onde ele importa. Células claramente interiores ou exteriores são representadas por um único nó, independentemente de seu tamanho.

4.2 Definição Recursiva

Definição 4.1 (Quadtree). Uma Quadtree sobre uma região retangular R e uma forma implícita f é uma árvore quaternária definida recursivamente:

- A *raiz* representa a região R inteira.
- Se um nó v (representando a sub-região R_v) tem Status 2 e a profundidade de v é menor que $d_{\max}$, ele é *subdividido*: R_v é particionada em quatro quadrantes iguais $R_{v,1}, R_{v,2}, R_{v,3}, R_{v,4}$, que se tornam os quatro filhos de v .
- Nós com Status 0 ou Status 1, ou nós em profundidade $d_{\max}$, são *folhas*.

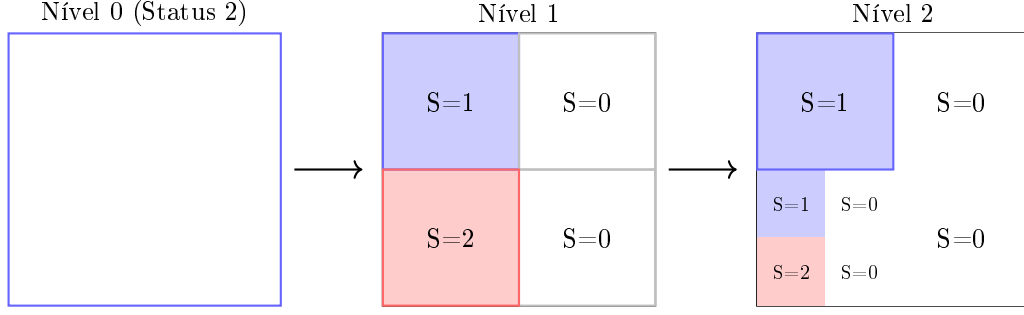


Figura 1: Construção iterativa da Quadtree. Apenas células com Status 2 (vermelho) são subdivididas. Células com Status 0 (branco) ou Status 1 (azul) são folhas.

4.3 Algoritmo de Construção

O algoritmo de construção é uma busca em profundidade (*DFS*) que constrói a árvore top-down:

```

procedure Build( $R, d, d_{\max}, f$ )
     $s \leftarrow \text{ClassifyCell}(R, f)$ 
     $v \leftarrow \text{NovoNó}(R, s)$ 
    if  $s = 2$  and  $d < d_{\max}$  then
         $(R_1, R_2, R_3, R_4) \leftarrow \text{Subdividir}(R)$ 
        for  $i = 1, 2, 3, 4$  do
             $v.\text{filhos}[i] \leftarrow \text{BUILD}(R_i, d + 1, d_{\max}, f)$ 
        end for
    end if
    return  $v$ 
end procedure

```

Algoritmo 1: Construção da Quadtree

A subdivisão de um nó com região $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ produz quatro filhos:

$$\begin{aligned}
 R_1 &= [x_{\min}, x_c] \times [y_{\min}, y_c] && \text{(inferior esquerdo)} \\
 R_2 &= [x_c, x_{\max}] \times [y_{\min}, y_c] && \text{(inferior direito)} \\
 R_3 &= [x_c, x_{\max}] \times [y_c, y_{\max}] && \text{(superior direito)} \\
 R_4 &= [x_{\min}, x_c] \times [y_c, y_{\max}] && \text{(superior esquerdo)}
 \end{aligned}$$

onde $x_c = (x_{\min} + x_{\max})/2$ e $y_c = (y_{\min} + y_{\max})/2$.

4.3.1 Complexidade

Proposição 4.2 (Complexidade da Quadtree). *Seja L o número de células de fronteira (Status 2) na resolução máxima $d_{\max}$. Então a Quadtree tem $O(L \cdot d_{\max})$ nós no total.*

Na prática, para uma curva suave, $L = O(W \cdot H/2^{d_{\max}})$ cresce linearmente com o perímetro discreto, e a Quadtree representa a fronteira com muito menos memória do que uma grade uniforme de resolução equivalente.

4.4 Poda (Pruning)

Motivação

A poda pós-construção tem dois objetivos: (i) reduzir o uso de memória eliminando subdivisões desnecessárias; e (ii) acelerar a travessia de raios. No Ray Tracing, quando um raio entra em um nó folha com Status 0 (vazio), podemos imediatamente descartá-lo sem processar filhos. A poda garante que não existam nós com filhos redundantes — um nó pai com quatro filhos todos do mesmo status é equivalente a uma única folha daquele status.

Definição 4.3 (Poda de Quadtree). Dado um nó v com filhos v_1, v_2, v_3, v_4 , se $\text{status}(v_1) = \text{status}(v_2) = \text{status}(v_3) = \text{status}(v_4) = s$ para algum $s \in \{0, 1\}$, então descartamos os filhos e definimos $\text{status}(v) \leftarrow s$. O nó v se torna uma folha.

O algoritmo de poda é um percurso em pós-ordem (*post-order DFS*): primeiro processamos todos os filhos recursivamente, depois verificamos se o nó pai pode ser podado.

```
procedure Prune( $v$ )
if  $v$  é folha then return
end if
for cada filho  $c$  de  $v$  do
    PRUNE( $c$ )
end for
 $s_0 \leftarrow \text{status}(v.\text{filho}_1)$ 
if todos os filhos têm o mesmo status  $s_0$  and  $s_0 \neq 2$  then
     $v.\text{filhos} \leftarrow \emptyset$ 
     $v.\text{status} \leftarrow s_0$ 
end if
end procedure
```

*descarta os filhos
v vira folha*

Algoritmo 2: Poda da Quadtree (post-order)

Observação 4.4. A poda só é aplicada para Status 0 e 1, nunca para Status 2. Um nó com quatro filhos de fronteira (Status 2) *não* pode ser fundido, pois isso perderia informação sobre onde a fronteira está localizada dentro de cada filho.

5 Ray Tracing Volumétrico

5.1 Modelo de Iluminação e Motivação

Motivação

Um renderizador baseado em *ray casting* simples lançaria um raio de cada pixel até a cena, determinando sua cor. Aqui invertemos a lógica: lançamos raios *a partir da fonte de luz* em todas as direções. Cada raio carrega a luz da lanterna até encontrar uma parede (fronteira da forma implícita) ou atingir o limite de alcance. Os pixels iluminados por pelo menos um raio ficam visíveis; os demais permanecem no escuro. Isso é eficiente para simular uma única fonte de luz direcional com sombras corretas.

5.2 Equação do Raio

Um raio é parametrizado por:

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \quad t \geq 0,$$

onde $\mathbf{o} = (o_x, o_y)$ é a origem (posição da lanterna) e $\mathbf{d} = (d_x, d_y)$ é o vetor direção unitário ($\|\mathbf{d}\| = 1$). Para cada ângulo $\theta_i = i \cdot \frac{2\pi}{N_{\text{raios}}}$, lançamos um raio com direção

$$\mathbf{d}_i = (\cos \theta_i, \sin \theta_i).$$

5.3 Interseção Raio–AABB: O Método das Lâminas

Para percorrer a Quadtree eficientemente, precisamos determinar se um raio intersecta um nó (uma caixa delimitadora alinhada aos eixos, AABB). O método das lâminas (*slab method*) resolve isso de forma eficiente.

Definição 5.1 (AABB). Uma AABB em 2D é um retângulo alinhado aos eixos: $B = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$.

Teorema 5.2 (Interseção Raio-AABB). O raio $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ intersecta a AABB B se e somente se o intervalo de interseção $[t_{\min}, t_{\max}]$, definido como abaixo, satisfaz $t_{\max} \geq t_{\min}$ e $t_{\max} \geq 0$.

Demonstração. Para cada eixo $k \in \{x, y\}$, o raio está dentro da “lâmina” $[B_{k,\min}, B_{k,\max}]$ para $t \in [t_{k,1}, t_{k,2}]$, onde

$$t_{k,1} = \frac{B_{k,\min} - o_k}{d_k}, \quad t_{k,2} = \frac{B_{k,\max} - o_k}{d_k}.$$

(Se $d_k = 0$, o raio é paralelo ao eixo: está dentro da lâmina se e somente se $o_k \in [B_{k,\min}, B_{k,\max}]$, caso contrário não há interseção.)

O raio está *simultaneamente* dentro de todas as lâminas para $t \in [t_{\min}, t_{\max}]$, onde

$$t_{\min} = \max_k \min(t_{k,1}, t_{k,2}), \quad t_{\max} = \min_k \max(t_{k,1}, t_{k,2}).$$

A condição $t_{\max} \geq t_{\min}$ garante que este intervalo é não-vazio (o raio está em todas as lâminas ao mesmo tempo). A condição $t_{\max} \geq 0$ garante que a interseção ocorre à frente da origem. $\square$

A implementação direta:

```

1 rayIntersectAABB(ox, oy, dx, dy, xMin, yMin, xMax, yMax) {
2   let tmin = -Infinity, tmax = Infinity;
3
4   // Lamina do eixo X
5   if (Math.abs(dx) > 1e-6) {
6     let tx1 = (xMin - ox) / dx;
7     let tx2 = (xMax - ox) / dx;
8     tmin = Math.max(tmin, Math.min(tx1, tx2));
9     tmax = Math.min(tmax, Math.max(tx1, tx2));
10  } else if (ox < xMin || ox > xMax) return null; // Paralelo e fora
11
12  // Lamina do eixo Y
13  if (Math.abs(dy) > 1e-6) {
14    let ty1 = (yMin - oy) / dy;
15    let ty2 = (yMax - oy) / dy;
16    tmin = Math.max(tmin, Math.min(ty1, ty2));
17    tmax = Math.min(tmax, Math.max(ty1, ty2));
18  } else if (oy < yMin || oy > yMax) return null;
19
20  if (tmax >= tmin && tmax >= 0)
21    return { tmin: Math.max(0, tmin), tmax };
22  return null;
23 }
```

5.4 Travessia Hierárquica da Quadtree

Com o teste de interseção raio-AABB em mãos, podemos percorrer a Quadtree de forma eficiente. A ideia central é a **poda por visibilidade**: se um raio não intersecta a AABB de um nó, ele não pode intersectar nenhum descendente daquele nó, pois as AABBs dos filhos estão contidas na do pai.

Além disso, exploramos o conhecimento sobre o **estado da luz**: se a lanterna está *fora* dos objetos ($f(\mathbf{o}) > 0$), os raios viajam livremente pelo exterior (Status 0) e só colidem ao tentar entrar em uma forma ($f \leq 0$). Portanto, podemos **ignorar completamente** nós com Status 0. Simetricamente, se a luz está *dentro*, ignoramos nós com Status 1.

```

procedure Trace( $v$ ,  $\mathbf{o}$ ,  $\mathbf{d}$ , isInside, scene)
if  $v$  é nulo then return null
end if
if isInside and  $v$ .status = 1 then return null
end if Luz viaja livre no interior
if not isInside and  $v$ .status = 0 then return null
end if Luz viaja livre no exterior
if Raio não intersecta AABB de  $v$  then return null
end if
if  $v$  tem filhos then
    Calcula  $t_{\min}$  de interseção de cada filho com o raio
    Ordena filhos por  $t_{\min}$  crescente Nearest-first
for cada filho  $c$  em ordem do
    resultado  $\leftarrow$  TRACE( $c$ ,  $\mathbf{o}$ ,  $\mathbf{d}$ , isInside, scene)
if resultado  $\neq$  null then return resultado
end if
end for return null
else Nó folha de fronteira: raymarching fino
    return RAYMARCH( $v$ ,  $\mathbf{o}$ ,  $\mathbf{d}$ , isInside, scene)
end if
end procedure

```

Algoritmo 3: Travessia de Raio na Quadtree

Detalhe de Implementação

Por que ordenar os filhos por $t_{\min}$? Na travessia, assim que encontramos a *primeira* colisão, podemos retornar imediatamente (*early exit*). Visitando os filhos em ordem crescente de $t_{\min}$ (da esquerda para a direita ao longo do raio), garantimos que a primeira colisão encontrada é de fato a mais próxima da origem. Sem essa ordenação, poderíamos reportar uma colisão mais distante antes de uma mais próxima, produzindo sombras incorretas.

5.5 Raymarching na Fronteira

Ao atingir um nó folha com Status 2 (fronteira), não temos a equação analítica da interseção raio-forma. A solução é o **raymarching**: amostramos a função f em passos discretos ao longo do raio e detectamos a mudança de sinal.

Para um nó com intervalo de interseção $[t_{\min}, t_{\max}]$, iteramos:

$$t_k = t_{\min} + k \cdot \Delta t, \quad k = 0, 1, 2, \dots$$

e avaliamos $f(\mathbf{r}(t_k)) = f(o_x + d_x t_k, o_y + d_y t_k)$.

A colisão é detectada por uma *mudança de sinal* que depende do estado:

$$\begin{aligned} \text{(luz fora): } f(\mathbf{r}(t_k)) &\leq 0 \Rightarrow \text{hit em } t_k \\ \text{(luz dentro): } f(\mathbf{r}(t_k)) &> 0 \Rightarrow \text{hit em } t_k \end{aligned}$$

O passo $\Delta t = 1.0$ pixel foi escolhido para que nenhuma fronteira seja “pulada” — como a fronteira tem, no mínimo, 1 pixel de espessura na resolução do canvas, um passo de 1 pixel garante que ela será detectada.

Observação 5.3. O raymarching é executado *somente dentro de nós folha de Status 2*. O resto da árvore é percorrido analiticamente pelo teste AABB. Isso é a essência da aceleração: trocamos o teste analítico barato (AABB) pelo teste caro (avaliação de f) apenas onde é absolutamente necessário.

5.6 Detecção de Imersão e Iluminação Invertida

Um recurso elegante do sistema é o suporte a uma lanterna dentro de um objeto. O estado de imersão é determinado avaliando a cena na posição da lanterna:

$$\text{isInside} = (f_{\text{scene}}(o_x, o_y) \leq 0).$$

Quando `isInside = true`:

- A cor base dos raios muda de amarela (255, 245, 220) para azul (74, 144, 217).
- O raio viaja pelo interior (azul na Quadtree) sem obstrução.
- A colisão ocorre quando o raio *tenta sair* do objeto ($f > 0$).
- O resultado visual é que o interior do objeto é iluminado, enquanto o exterior permanece no escuro — como uma luz dentro de uma bolha de vidro.

5.7 Modelo de Decaimento de Luz

A intensidade da luz decai com a distância. Usamos uma função de decaimento **polinomial** de expoente $\alpha = 2.5$:

$$I(t) = I_0 \cdot \left(1 - \frac{t}{R}\right)^\alpha, \quad 0 \leq t \leq R,$$

onde R é o alcance máximo da lanterna (parâmetro `lightRange`) e I_0 é a intensidade na origem.

Motivação

Por que $\alpha = 2.5$ em vez do decaimento quadrático inverso da lei física $I \propto 1/t^2$? A lei $1/t^2$ produz um pico extremamente brilhante próximo à origem e escuridão abrupta à distância, o que é visualmente agressivo em tempo real. A função $(1 - t/R)^{2.5}$ garante que $I(R) = 0$ (a luz se apaga suavemente no limite do alcance), produz um núcleo brilhante e uma queda gradual que o olho humano percebe como “natural”. O expoente 2.5 foi escolhido empiricamente como um bom compromisso entre brilho central e suavidade nas bordas.

Para renderizar o gradiente, o sistema cria um gradiente linear do Canvas 2D da origem até o ponto final do raio, com quatro paradas:

Posição	Opacidade	Correspondência à fórmula
$t/R = 0$	1.0	$I(0) = I_0$
$t/R = 0.15$	0.8	$I(0.15R) \approx 0.75I_0$
$t/R = 0.5$	0.25	$I(0.5R) \approx 0.18I_0$
$t/R = 1.0$	0.0	$I(R) = 0$

5.8 Visualização do Modo Adaptativo

No modo de visualização “Estrutura Quadtree”, o sistema usa o `AdaptiveQuadtreeAnimator` para revelar progressivamente a estrutura da árvore, pintando cada nó com a cor correspondente ao seu status. A revelação segue uma travessia BFS (largura), tornando visível a progressão hierárquica do nível raiz até as folhas mais refinadas.

A animação funciona mantendo uma `ImageData` (buffer de pixels bruto) que é atualizada incrementalmente a cada frame. O método `step(nowMs)` calcula quantos nós revelar com base no tempo decorrido, garantindo que a velocidade de revelação seja independente da taxa de frames:

$$\Delta n = \left\lfloor \frac{\Delta t_{\text{elapsed}}}{\Delta t_{\text{step}}} \right\rfloor,$$

onde Δt_{step} é o intervalo mínimo entre revelações (30ms no sistema atual).

6 Física de Partículas e Colisões Implícitas

6.1 Modelo de Movimento

Cada partícula é um ponto com posição $\mathbf{p} = (x, y)$ e velocidade $\mathbf{v} = (v_x, v_y)$. A integração de Euler explícita atualiza o estado a cada frame:

$$\mathbf{p}^{(n+1)} = \mathbf{p}^{(n)} + \Delta t \cdot \mathbf{v}^{(n)},$$

onde o passo de tempo Δt é implicitamente 1 frame (a velocidade é expressa em pixels/frame).

As paredes do canvas são tratadas como bordas rígidas: ao ultrapassar qualquer borda, a componente correspondente da velocidade é invertida.

6.2 Colisão com Formas Implícitas: Reflexão Pelo Gradiente

Motivação

Como fazer uma partícula “quicar” numa forma implícita arbitrária, sem conhecer sua geometria explicitamente? O gradiente ∇f aponta na direção de maior crescimento de f , que é perpendicular à curva $f = 0$. Portanto, $\hat{\mathbf{n}} = \nabla f / \|\nabla f\|$ é a **normal** à superfície implícita. Com a normal, podemos aplicar a lei clássica de reflexão especular para calcular o novo vetor velocidade após a colisão.

Quando a posição tentativa $\mathbf{p}^* = \mathbf{p} + \mathbf{v}$ cai dentro de uma forma ($f(\mathbf{p}^*) \leq 0$), a colisão é detectada. O gradiente é aproximado por diferenças finitas centrais com passo $\varepsilon = 1.0$ pixel:

$$\frac{\partial f}{\partial x}(\mathbf{p}^*) \approx \frac{f(p_x^* + \varepsilon, p_y^*) - f(p_x^* - \varepsilon, p_y^*)}{2\varepsilon} \quad (3)$$

$$\frac{\partial f}{\partial y}(\mathbf{p}^*) \approx \frac{f(p_x^*, p_y^* + \varepsilon) - f(p_x^*, p_y^* - \varepsilon)}{2\varepsilon} \quad (4)$$

Como ε é constante e cancela na normalização do vetor, simplificamos calculando apenas o numerador:

$$\mathbf{g} = (f(p_x^* + \varepsilon, p_y^*) - f(p_x^* - \varepsilon, p_y^*), f(p_x^*, p_y^* + \varepsilon) - f(p_x^*, p_y^* - \varepsilon)).$$

A normal unitária é $\hat{\mathbf{n}} = \mathbf{g} / \|\mathbf{g}\|$. A velocidade refletida é dada pela fórmula de reflexão especular:

$$\mathbf{v}' = \mathbf{v} - 2(\mathbf{v} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}. \quad (5)$$

Dedução da Fórmula de Reflexão. Decompomos $\mathbf{v}$ em componentes normal e tangencial à superfície:

$$\mathbf{v} = \underbrace{(\mathbf{v} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}}_{\mathbf{v}_n} + \underbrace{(\mathbf{v} - (\mathbf{v} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}})}_{\mathbf{v}_t}.$$

Na reflexão elástica, a componente tangencial é preservada e a componente normal é invertida: $\mathbf{v}'_n = -\mathbf{v}_n$, $\mathbf{v}'_t = \mathbf{v}_t$. Portanto:

$$\mathbf{v}' = \mathbf{v}_t + \mathbf{v}'_n = \mathbf{v}_t - \mathbf{v}_n = \mathbf{v} - 2\mathbf{v}_n = \mathbf{v} - 2(\mathbf{v} \cdot \hat{\mathbf{n}})\hat{\mathbf{n}}.$$

□

A implementação em JavaScript:

```

1 update(scene, w, h) {
2   let nextX = this.x + this.vx;
3   let nextY = this.y + this.vy;
4
5   if (scene.evaluate(nextX, nextY) <= 0) {
6     const eps = 1.0;
7     // Gradiente por diferencas finitas (eq. 6.3 e 6.4)
8     const nx = scene.evaluate(nextX+eps, nextY)
9       - scene.evaluate(nextX-eps, nextY);
10    const ny = scene.evaluate(nextX, nextY+eps)
11      - scene.evaluate(nextX, nextY-eps);
12    const len = Math.sqrt(nx*nx + ny*ny) || 1; // ||grad f||
13
14    // Formula de reflexao (eq. 6.5)
15    const dot = (this.vx*nx + this.vy*ny) / len;
16    this.vx -= 2 * dot * (nx / len);
17    this.vy -= 2 * dot * (ny / len);
18    // Nao avanca a posicao (permanece fora da forma)
19  } else {
20    this.x = nextX;
21    this.y = nextY;
22  }
23 }
```

6.3 Sombras Dinâmicas por Shadow Ray

Para cada partícula visível, verificamos se ela está na “linha de visão” da lanterna. Isso é feito lançando um **shadow ray** da lanterna em direção à partícula.

Seja $\mathbf{o}$ a posição da lanterna, $\mathbf{p}$ a posição da partícula e $d_{\mathbf{op}} = \|\mathbf{p} - \mathbf{o}\|$ a distância entre elas. O shadow ray tem direção

$$\hat{\mathbf{d}} = \frac{\mathbf{p} - \mathbf{o}}{d_{\mathbf{op}}}.$$

A partícula está em sombra se e somente se o ray tracer detecta uma colisão a uma distância $t_{\text{hit}} < d_{\mathbf{op}} - r_{\text{partícula}}$ — a subtração de $r_{\text{partícula}}$ evita que a partícula faça sombra sobre si mesma (*self-shadowing*).

Se a partícula não está em sombra, sua opacidade visual é modulada pelo mesmo modelo de decaimento da Seção 5.7:

$$\alpha_{\text{halo}} = \left(1 - \frac{d_{\text{op}}}{R}\right)^{2.5},$$

criando um efeito de “partícula dentro do cone de luz”.

7 Análise de Complexidade e Performance

7.1 Custo do Ray Tracing por Frame

Seja N o número de raios, d_{max} a profundidade da Quadtree e L o número médio de nós folha de Status 2 intersectados por um raio. O custo de um frame é:

Etapa	Custo por raio	Custo total
Teste AABB nos nós internos	$O(d_{\text{max}})$	$O(N \cdot d_{\text{max}})$
Raymarching nos nós folha	$O(L \cdot s)$	$O(N \cdot L \cdot s)$
Gradiente (partículas)	$O(P)$	$O(P)$ por frame
Shadow rays (partículas)	$O(d_{\text{max}})$ cada	$O(P \cdot d_{\text{max}})$

onde $s = 2^{-d_{\text{max}}} \cdot \max(W, H)$ é o tamanho do nó folha em pixels e P é o número de partículas. Para os parâmetros padrões do sistema ($N = 720$, $d_{\text{max}} = 7$, $W = H = 700$), o custo por frame é dominado pelo raymarching, mas permanece tratável em tempo real (60fps).

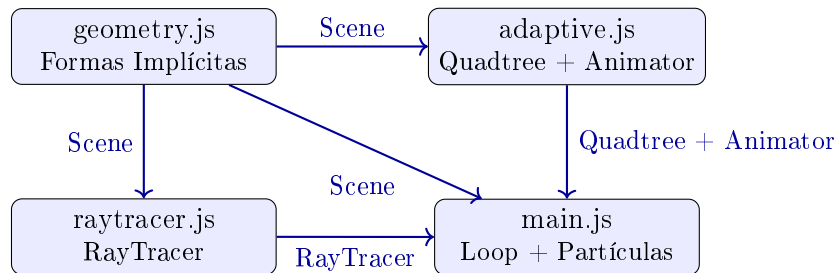
7.2 Vantagem da Poda

Sem poda, a Quadtree teria $4^{d_{\text{max}}}$ nós folha no pior caso. Com poda, regiões uniformes (Status 0 ou 1) são colapsadas em um único nó. Para uma forma circular em um canvas vazio, aproximadamente 75% dos nós da Quadtree não refinada seriam de Status 0 (exterior) e seriam podados, reduzindo drasticamente o número de testes AABB por raio.

8 Integração do Sistema: o Loop Principal

8.1 Arquitetura de Módulos

O sistema é organizado em quatro módulos JavaScript com responsabilidades bem delimitadas:



8.2 Sequência de Operações por Frame

A cada frame do `requestAnimationFrame`, o loop principal executa:

1. **Leitura dos controles:** atualiza `lightRange` e `numRays` a partir dos sliders.
2. **Renderização do fundo:**

- *Modo Lanterna:* `RayTracer.renderLight()` lança N raios e pinta os gradientes de luz.
 - *Modo Quadtree:* `AdaptiveQuadtreeAnimator.step()` e `.draw()` revelam progressivamente a estrutura, seguidos da sobreposição dos contornos de fronteira.
3. **Física de partículas:** para cada partícula, `update()` integra a posição e resolve colisões.
 4. **Renderização de partículas:** para cada partícula, `draw()` lança o shadow ray e, se visível, pinta o halo e o núcleo.

8.3 Evento rebuild()

O evento `rebuild()` é disparado sempre que a cena muda (adição de forma, limpeza, mudança de profundidade). Ele:

1. Reconstrói a `Quadtree` do zero com a cena atual.
2. Chama `prune()` para otimizar a árvore.
3. Reinicia o `AdaptiveQuadtreeAnimator` com a nova árvore.

Isso garante que a `Quadtree` usada pelo Ray Tracer e pelo Animator é sempre consistente com a cena em exibição.

9 Funções Customizadas: da String ao Código

O sistema permite que o usuário insira uma expressão matemática como string — por exemplo, `"x*x + y*y - 0.25"` — e a utilize como função implícita. A transformação ocorre em dois passos:

Passo 1: Compilação. A string é convertida em uma função JavaScript usando o construtor `Function`:

```
1 this.func = new Function('x', 'y', 'return ${exprStr};');
```

Isso cria uma função anônima equivalente a $(x, y) \Rightarrow \text{exprStr}$. O motor V8 do Chrome/Node compila essa função para bytecode nativo, garantindo performance equivalente a código escrito manualmente.

Passo 2: Avaliação no domínio correto. Cada chamada a `evaluate(px, py)` primeiro converte as coordenadas de pixel para o domínio $[-2, 2]$ (equações (1) e (2)) antes de chamar `this.func`.

Exemplo 9.1 (Equações que podem ser inseridas). • $x*x + y*y - 0.25$ — círculo de raio 0.5 centrado na origem.

- $y - x*x$ — parábola $y = x^2$ (interior abaixo da curva).
- $x*x/0.25 + y*y/1.0 - 1$ — elipse com semi-eixos $a = 0.5$, $b = 1$.
- $\text{Math.sin}(x*3) - y$ — curva senoidal.
- $\text{Math.abs}(x) + \text{Math.abs}(y) - 0.5$ — quadrado rotacionado 45° (losango).

10 Conclusão

Este trabalho demonstrou como conceitos fundamentais de matemática aplicada se traduzem diretamente em sistemas computacionais funcionais e eficientes:

- A **geometria implícita** fornece o substrato matemático unificado que permite descrever formas arbitrárias, testá-las por inclusão em $O(1)$ e combiná-las por operações de conjuntos (min/max).
- A **Quadtree adaptativa** é a estrutura de dados que torna o sistema escalável: concentra representação e computação na fronteira das formas, onde o comportamento é complexo, e descarta regiões uniformes com custo constante.
- O **Ray Tracing** combina a elegância do método das lâminas para teste AABB com o pragmatismo do raymarching para localizar fronteiras implícitas com precisão sub-pixel, tudo acelerado pela hierarquia da Quadtree.
- A **física de partículas** demonstra que o gradiente de uma função implícita é não apenas um conceito teórico de análise, mas uma ferramenta prática imediata para computar normais e resolver reflexões sem qualquer conhecimento explícito da geometria.

A integração desses quatro pilares num loop de tempo real — mantendo 60fps no navegador com centenas de partículas e centenas de raios — valida a premissa central: a modelagem matemática rigorosa não é antagonista do desempenho computacional, mas sua pré-condição.

A Pseudocódigo Completo do Loop Principal

```
procedure Loop(nowMs)
  Atualiza  $R \leftarrow \text{lightRange}$ ,  $N \leftarrow \text{numRays}$ 
if modo = LANTERNA then
  Pinta fundo preto
  isInside  $\leftarrow f_{\text{scene}}(\mathbf{o}) \leq 0$ 
for  $i = 0$  to  $N - 1$  do
   $\theta_i \leftarrow 2\pi i / N$ 
   $\mathbf{d} \leftarrow (\cos \theta_i, \sin \theta_i)$ 
  hit  $\leftarrow \text{TRACE}(\text{root}, \mathbf{o}, \mathbf{d}, \text{isInside}, \text{scene})$ 
   $t_{\text{end}} \leftarrow \min(\text{hit.t se hit}, R)$ 
  Pinta gradiente de  $\mathbf{o}$  até  $\mathbf{o} + t_{\text{end}}\mathbf{d}$ 
end for
else Modo Quadtree
  ANIMATORSTEP(nowMs)
  ANIMATORDRAW(ctx)
  Sobrepõe contornos de nós Status 2
end if
for cada partícula  $p$  do
  UPDATE( $p$ , scene)
  DRAWPARTICLE( $p$ ,  $\mathbf{o}$ ,  $R$ , scene, quadtree)
end for
  requestAnimationFrame(LOOP())
end procedure
```

Algoritmo 4: Loop Principal de Renderização

B Tabela de Parâmetros do Sistema

Parâmetro	Símbolo	Padrão	Intervalo
Número de raios	N	720	[90, 1440]
Profundidade da Quadtree	$d_{\max}$	7	[2, 10]
Alcance da lanterna	R	300px	[50, 600]px
Expoente de decaimento	α	2.5	(fixo)
Raio do círculo padrão	r	0.08	(fixo)
Passo do raymarching	Δt	1.0px	(fixo)
Passo do gradiente	ε	1.0px	(fixo)
Velocidade da partícula	$\ \mathbf{v}\ $	$\sim 4.2\text{px/frame}$	(aleatório)

Referências

- [1] GOMES, Jonas; VELHO, Luiz.: **Fundamentos da Computação Gráfica** (Foco em modelagem geométrica e síntese de imagem).
- [2] SHIRLEY, Peter; MARSHNER, Steve.: **Fundamentals of Computer Graphics**. A. K. Peters/CRC Press, (Referência para fundamentação de Ray Tracing).
- [3] PREPARATA, F. P; SHAMOS, M I. (1985).: **Computational Geometry: An Introduction**". Springer-Verlag
- [4] SAMET, H. (1984) **The Quadtree and Related Hierarchical Data Structures**. ACM Computing Surveys
- [5] PHARR, Matt.; JAKOB. Wenzel; HUMPHREYS. Greg **Physically Based Rendering: From Theory To Implementation**. Morgan Kaufmann. (Teoria avançada de modelagem para transporte de luz).
- [6] VISGRAF Lab. **Topologia e Computação Visual 2026**. Disponível em: <<https://visgraf.github.io/tcv-2026/>>. Acesso em: 14 mai. 2026.